

# Ask or Assume? Uncertainty-Aware Clarification-Seeking in Coding Agents

Nicholas Edwards<sup>1,2</sup>

Sebastian Schuster<sup>1</sup>

<sup>1</sup>Faculty of Computer Science, University of Vienna, Vienna, Austria

<sup>2</sup>UniVie Doctoral School Computer Science, University of Vienna, Vienna, Austria  
{nicholas.edwards, sebastian.schuster}@univie.ac.at

## Abstract

As Large Language Model (LLM) agents are increasingly deployed in open-ended domains like software engineering, they frequently encounter *underspecified* instructions that lack crucial context. While human developers naturally resolve underspecification by asking clarifying questions, current agents are largely optimized for autonomous execution. In this work, we systematically evaluate the clarification-seeking abilities of LLM agents on an underspecified variant of SWE-bench Verified. We propose an uncertainty-aware multi-agent scaffold that decouples underspecification detection from code execution. Across both proprietary and open-weight frontier LLMs, our scaffold achieves a 69.40% task resolve rate, significantly outperforming a standard single-agent setup and closing the performance gap with agents operating on fully specified instructions. Furthermore, we find that the multi-agent system exhibits well-calibrated information-seeking behavior, conserving queries on simple tasks while proactively seeking information on more complex issues. These findings indicate that current models can be turned into proactive collaborators, where agents independently recognize when to ask questions to elicit missing information in real-world, underspecified tasks.

## 1 Introduction

A core property of a good collaborator is the ability to establish and maintain a shared understanding of a goal (Clark and Wilkes-Gibbs, 1986). In human communication, this is rarely achieved through a single perfectly specified instruction; instead, interlocutors infer missing context and explicitly signal uncertainty, such as by asking clarification questions (Clark, 1996; Hawkins et al., 2015).

While information gathering in response to underspecification has been studied in the NLP domain, such as in question answering and conversational search (Rao and Daumé III, 2018; Alian-nejadi et al., 2019; Zhang and Choi, 2025), these

settings involve static or single-turn interactions over bounded contexts.

In contrast, modern AI agents are increasingly deployed in more naturalistic, open-ended domains such as software engineering. For instance, fixing real-world GitHub issues (Jimenez et al., 2024) requires exploring, understanding, and editing large-scale repositories often containing hundreds of files and thousands of lines of code, while remaining aligned with implicit developer intentions. Existing agents are optimized for autonomous completion rather than interactive collaboration, creating a critical gap between user intent and agent execution (METR, 2025; Shen et al., 2026; Wang et al., 2026).

Given these constraints, we investigate the abilities of LLMs to assess uncertainty and determine how and when to seek information. Specifically, we consider how models handle *external uncertainty*: uncertainty arising from ambiguous or underspecified task inputs, as opposed to internal uncertainty inherent to the model itself (Li et al., 2026b). While LLMs can exhibit sensitivity to uncertainty (Kadavath et al., 2022), this is not always robust (Andukuri et al., 2024; Kapoor et al., 2024). Beyond simply detecting external uncertainty, a true agentic collaborator must be able to continuously monitor for such uncertainty and proactively initiate dialogue to elicit missing information. We define such a system as *uncertainty-aware*. Importantly, we use this term in a behavioral rather than formalized mathematical sense, evaluating how effectively an agent modulates its information-seeking behavior based on the severity of underspecification.

We systematically evaluate the clarification-seeking abilities of LLM agents within a dynamic, multi-turn software engineering framework. We conduct our evaluation using an underspecified variant of SWE-bench Verified (Chowdhury et al., 2024; Vijayvargiya et al., 2026b), where information is removed from the dataset’s original GitHub issues. To assess the capabilities of agents to act

as question-asking collaborators, we design agent scaffolds where models must rely on their own calibration of external uncertainty to decide when to query a user.

**Contributions** We develop and test both single- and multi-agent frameworks using frontier LLMs (Claude Sonnet 4.5 and Kimi K2.6) to assess how agents handle uncertainty arising from underspecified inputs. We find that interactive agents can successfully identify and retrieve missing information, obtaining a resolve rate comparable to an autonomous agent provided with a fully specified issue. Our results provide evidence that some models exhibit well-calibrated clarification-seeking behavior, accurately recognizing when an issue is already resolvable and refraining from unnecessary interaction. Our code is available at <https://github.com/nedwards99/ask-or-assume>.

## 2 Related Work

**Proactive Dialogue Agents** In human communication, interlocutors use clarification requests to establish shared understanding and resolve ambiguity (Clark and Wilkes-Gibbs, 1986). Recent work in the NLP domain has considered clarification-seeking behavior in the context of dialogue-based tasks like open-domain question-answering, conversational search and preference elicitation (Rao and Daumé III, 2018; Aliannejadi et al., 2019; Kim et al., 2024; Li et al., 2025; Zhang and Choi, 2025; Zhang et al., 2025).

To elicit these proactive clarification-seeking behaviors, prior work has explored various methodological approaches. For instance, Zhang et al. (2025) propose modeling the utility of future conversation trajectories to teach LLMs when a question is necessary; Andukuri et al. (2024) utilize simulated self-play to iteratively improve the model’s clarification skills; and Testoni and Fernández (2024) introduce an uncertainty-guided approach, demonstrating that leveraging a model’s own uncertainty (operationalized as predictive entropy) to trigger questions is more effective than imitating human clarification-seeking behavior. While these methods have proven effective in bounded conversational settings, our work extends this paradigm by evaluating how agents dynamically elicit missing information in open-ended, multi-step software engineering environments.

**Agentic Systems for Ambiguous Code Generation** As LLM agents are increasingly deployed in software engineering, robustly handling ambiguous or underspecified instructions presents a major challenge. Existing coding environments used to evaluate clarification-seeking behaviors often focus on tasks targeting edits to isolated functions, typically allowing only a single, predetermined round of clarification or short, constrained exchanges (Li et al., 2023; Mu et al., 2024; Wu and Fard, 2025; Li et al., 2026a).

To improve the clarification-seeking behavior of agents in dynamic, multi-turn coding settings, recent literature has introduced training pipelines leveraging reinforcement learning (Sun et al., 2026) and contrastive training (Chen et al., 2025) techniques. However, these frameworks often impose rigid interaction structures. For example, both Vijayvargiya et al. (2026a) and Darji and Lutellier (2025) strictly constrain question-asking to the very beginning of the trajectory, deferring all code execution until the initial prompt is disambiguated.

In practice, relevant missing information may only become apparent after initial exploration of a codebase. Most closely related to our evaluation setting, Vijayvargiya et al. (2026b) analyze how LLM agents handle underspecification in software engineering tasks. In our work, we demonstrate that a specialized multi-agent scaffold that decouples underspecification detection from code execution enables agents to navigate underspecified tasks and actively ask questions throughout a trajectory.

**Uncertainty Detection in LLMs** A prerequisite for proactive clarification is an agent’s ability to accurately gauge its own uncertainty. Foundational work demonstrates that base LLMs exhibit a degree of internal calibration, enabling them to estimate the likelihood of their correctness or express their confidence in natural language (Kadavath et al., 2022; Lin et al., 2022). However, this calibration is not always robust out of the box (Kapoor et al., 2024; Andukuri et al., 2024), as standard post-training alignment techniques, such as RLHF, can inadvertently degrade calibration and encourage model overconfidence (Zhou et al., 2024). Furthermore, effective interaction requires distinguishing between different sources of uncertainty. For instance, recent work has highlighted the difference between uncertainty caused by a model’s lack of parametric knowledge versus input underspecification or ambiguity (Hou et al., 2024; Zhang and

Choi, 2025). Formally quantifying this distinction, Li et al. (2026b) distinguish *internal* uncertainty (inherent to the model) from *external* uncertainty (arising from ambiguous or underspecified inputs). Our work specifically investigates how agents handle external uncertainty. Rather than considering a formalized measure of confidence (e.g., a probability), our work evaluates uncertainty detection behaviorally. That is, we assess whether agents can reliably identify missing context and resolve it through asking the user for clarification.

### 3 Method

#### 3.1 Dataset and Evaluation Framework

We closely follow and adapt the interactive evaluation setting introduced by Vijayvargiya et al. (2026b). We use their dataset (CMU LTI, 2024), which is an underspecified variant of SWE-bench Verified (Chowdhury et al., 2024), a human-annotated subset of 500 GitHub issues derived from the original SWE-bench dataset (Jimenez et al., 2024). To construct their dataset, the authors utilized GPT-4o (Hurst et al., 2024) to summarize issues into underspecified variants, removing important details while preserving repository terminology. Given the automatic nature of the construction process, we performed an additional spot-check analysis to verify dataset validity (Appendix A); concrete examples are provided in Appendix D.

#### 3.2 Agent Design

**Agent Framework** We use the OpenHands (Wang et al., 2025) agent framework for all experiments. This framework enables an LLM to iteratively write, execute, and debug code within a secure sandbox environment, using tools for navigating codebases, editing files and executing Bash/Python scripts. To ensure comparability with Vijayvargiya et al. (2026b), all agent setups are provided with a maximum of 100 iterations.

**Agent Backbones** We evaluate two frontier backbone LLMs: Claude Sonnet 4.5 (Anthropic, 2025), a proprietary model, and Kimi K2.6 (Moonshot AI, 2026), an open-weight model. Both perform competitively in many coding benchmarks, serving as strong proxies for current agentic capabilities in interactive software engineering settings.

**User Simulator** Following recent interactive environments that utilize “oracle” user simulators with access to complete information (e.g., Yao

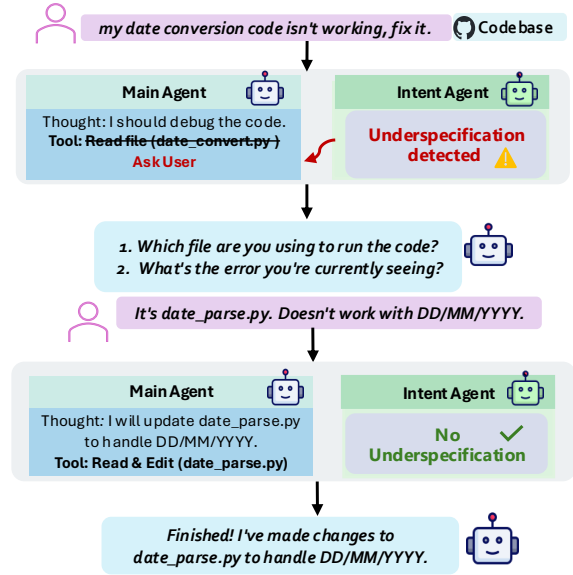


Figure 1: Illustration of the uncertainty-aware multi-agent scaffold. The Intent Agent monitors the state history at each turn, halting execution to query the user if missing information is required.

et al., 2025; Zhou et al., 2025), we employ GPT-5.1 (OpenAI, 2025) as the simulated user for all interactive agent configurations. We select this model to maintain consistency with the interactive SWE-bench setting introduced by Vijayvargiya et al. (2026b). The simulated user is provided with the original, fully specified issue and is constrained to answer queries from the coding agent using only this withheld context. The user simulator prompt and all additional agent and task prompts are detailed in Appendices B.1–B.3.

#### 3.3 Task Design

##### 3.3.1 Baselines

Following Vijayvargiya et al. (2026b), we evaluate our approach on the SWE-bench Verified dataset against three distinct baseline configurations.

**FULL** This is the standard SWE-bench setting, where the agent is provided with a fully specified version of the GitHub issue. The agent is prohibited from interacting with a user, representing default fully autonomous agent behavior.

**HIDDEN** In this configuration, the agent is provided with an underspecified version of the GitHub issue where details are missing. As in the FULL baseline, the agent cannot interact with a user.

**INTERACTIVE BASELINE** The agent receives an underspecified version of the GitHub issue (as

with the HIDDEN baseline) but can interact with a simulated user who possesses the fully specified issue. Importantly, the task prompt is modified to explicitly inform the agent that the issue description is incomplete, making it compulsory to query the user before proceeding with any execution. Consequently, this hardcoded instruction forces a pre-determined conversational turn with the user rather than evaluating independent information-seeking behavior.

### 3.3.2 Uncertainty-Aware Agents

To investigate how agents can independently detect and resolve underspecification, we propose two scaffolds. We refer to these as “uncertainty-aware” since they are designed to adapt their information-seeking behavior based on the extent of missing context. While these agents can query the same simulated user as the INTERACTIVE BASELINE, they do not rely on a hardcoded interaction prompt, instead using the default SWE-bench task prompt. Because they receive no prior warning that the issue is underspecified, they must independently identify missing context and query the simulated user only when they determine it is necessary.

**UNCERTAINTY-AWARE (SINGLE)** In this configuration, a single coding agent is prompted at each turn to check for underspecification and, if detected, to query the user. We refer to the agent hereafter as UA-SINGLE.

**UNCERTAINTY-AWARE (MULTI)** We investigate whether leveraging a multi-agent scaffold can improve underspecification detection. By assigning specialized roles to multiple LLMs, multi-agent systems can tackle increasingly complex tasks (Hong et al., 2024; Qian et al., 2024; Park et al., 2023). As illustrated in Figure 1, to decouple code execution from underspecification detection, we design a multi-agent scaffold consisting of two agents. The **Main Agent** is equipped with standard tools to navigate repositories, edit files, and execute code. The **Intent Agent** monitors the state history at each turn to determine if the user’s intent or repository context contains missing information. The Intent Agent is provided with its own dedicated system prompt (detailed in Appendix B.2) and has access to a single tool that outputs a binary decision indicating whether clarification is required. Whenever the Intent Agent detects underspecification, the Main Agent’s next action is constrained to query the user. In our experiments, both agents

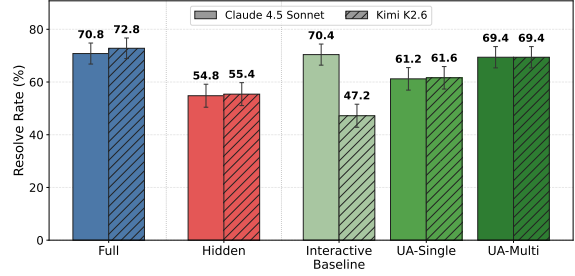


Figure 2: Task resolve rates (%) across evaluation settings for Claude Sonnet 4.5 (solid) and Kimi K2.6 (hatched), with 95% confidence intervals.

share the same LLM backbone. We refer to the agent hereafter as UA-MULTI.

## 4 Results and Discussion

Figure 2 presents the overall resolve rates.<sup>1</sup> For Claude Sonnet 4.5, UA-SINGLE outperforms the HIDDEN baseline (61.20% vs. 54.80%) but falls short of the prompted INTERACTIVE BASELINE (70.40%). However, separating underspecification detection and code execution in UA-MULTI substantially improves performance. It achieves a 69.40% resolve rate, yielding a significant improvement over UA-SINGLE ( $p < 0.001$ ) and closing the gap with the INTERACTIVE BASELINE ( $p = 0.621$ ) and FULL ( $p = 0.458$ ) configurations.

For Kimi K2.6, we observe similar trends, with UA-MULTI also achieving a 69.40% resolve rate, significantly improving over UA-SINGLE (61.60%,  $p < 0.001$ ), and approaching the performance of FULL, although a significant gap remains ( $p < 0.05$ ). A notable exception is the INTERACTIVE BASELINE, which achieves the lowest resolve rate of 47.20%. This is largely driven by instruction-following failures, where the model erroneously used the task completion tool to ask questions, highlighting the brittleness of hardcoded prompts and reliance on underlying model capability.

### Agent Uncertainty is Calibrated to Task Difficulty

The success of UA-MULTI can be largely attributed to its ability to discern *when* to ask questions. For Claude Sonnet 4.5, while it queried the user at least once in fewer overall tasks than

<sup>1</sup>Our reported resolve rates for the FULL baselines are slightly lower than the official OpenHands results (Claude Sonnet 4.5: 70.80% vs. 74.20%; Kimi K2.6: 72.80% vs. 74.60%). This is likely due to a combination of running for a maximum of 100 iterations instead of 500, as well as minor system prompt differences. Additionally, all reported  $p$ -values are computed via non-parametric permutation tests.

| Evaluation Setting                                                           | Resolve Rate (%) |              |
|------------------------------------------------------------------------------|------------------|--------------|
|                                                                              | Asked            | Did Not Ask  |
| <i>Claude Sonnet 4.5</i> ( $N_{\text{ask}} = 344$ , $N_{\text{not}} = 156$ ) |                  |              |
| UA-Multi                                                                     | 65.99            | 76.92        |
| Full                                                                         | <b>66.57</b>     | <b>80.13</b> |
| Hidden                                                                       | 44.48            | 77.56        |
| UA-Single                                                                    | 55.81            | 73.08        |
| Interactive Baseline                                                         | <b>66.57</b>     | 78.85        |
| <i>Kimi K2.6</i> ( $N_{\text{ask}} = 435$ , $N_{\text{not}} = 65$ )          |                  |              |
| UA-Multi                                                                     | 68.51            | 75.38        |
| Full                                                                         | <b>71.95</b>     | <b>78.46</b> |
| Hidden                                                                       | 52.18            | 76.92        |
| UA-Single                                                                    | 60.46            | 69.23        |
| Interactive Baseline                                                         | 45.29            | 60.00        |

Table 1: Resolve rates (%) conditioned on whether the UA-MULTI agent queried the user at least once in a task. Best performing settings per subset are in **bold**.

UA-SINGLE ( $N_{\text{ask}}=344$  vs.  $N_{\text{ask}}=369$ ), its interventions were substantially more effective, resolving 65.99% of queried tasks compared to UA-SINGLE’s 55.81% and the HIDDEN baseline’s 44.48%, while closely matching the INTERACTIVE BASELINE and FULL settings (both 66.57%). Crucially, the INTERACTIVE BASELINE achieves its high resolve rate at the cost of overclarification, asking questions in nearly every instance regardless of necessity. In contrast, UA-MULTI is highly selective in its interaction. For the 156 tasks where UA-MULTI refrained from asking, it still achieved a 76.92% resolve rate, closely matching the HIDDEN baseline (77.56%). Conversely, UA-MULTI with Kimi K2.6 queries more than UA-SINGLE ( $N_{\text{ask}}=435$  vs.  $N_{\text{ask}}=167$ ), but nevertheless achieves a competitive resolve rate on tasks where it refrains from asking (75.38%).

Appendix C.2 further validates query rate calibration across human-annotated task difficulty levels for SWE-bench Verified (Chowdhury et al., 2024), showing that UA-MULTI demonstrates an improved ability to distinguish when to ask based on task complexity. For Claude Sonnet 4.5, UA-MULTI exhibits a 9.28% higher ask rate for medium (“15 min – 1 hour”) tasks than easy (“<15 min”) tasks, compared to only a 2.43% increase for UA-SINGLE. For Kimi K2.6, both agents show similar easy-to-medium query rate increases, although UA-SINGLE’s ask rate declines for harder tasks, suggesting its query decisions do not reliably track task difficulty.

**Proactive Information Seeking** Beyond knowing *when* to ask, explicitly isolating the intent-

detection role improved *how* the agent interacted. For Claude Sonnet 4.5, UA-MULTI interrogated the user more iteratively, averaging 3.06 queries per task compared to 1.84 for UA-SINGLE, and distributed its queries across the early and middle stages of execution. For Kimi K2.6, UA-MULTI exhibits a much higher query rate, averaging 8.71 queries per task, with queries mainly concentrated in the middle stage of execution, suggesting poorer calibration relative to Claude Sonnet 4.5. A more detailed analysis is provided in Appendix C.1, along with qualitative trajectory comparisons in Appendix C.4.

## 5 Conclusion

We investigated the abilities of LLM agents to act as collaborators by independently identifying missing information and seeking clarification in underspecified software engineering tasks. We introduced an uncertainty-aware multi-agent scaffold that isolates the role of underspecification detection. Evaluated on an underspecified variant of SWE-bench Verified, this system achieved a 69.40% resolve rate across both proprietary and open-weight LLM backbones that effectively closed the performance gap with an autonomous agent operating on a fully specified issue.

Our results show that the multi-agent scaffold exhibits well-calibrated information-seeking behavior, accurately recognizing when an issue was already resolvable and refraining from unnecessary interaction on simpler tasks. Our results also reveal that underlying model capability shapes clarification behavior: while both LLM backbones benefit from the multi-agent scaffold, Kimi K2.6 exhibits higher query rates and poorer calibration, suggesting that robust uncertainty-aware interaction may require targeted training beyond architectural design alone. These findings present a promising step towards deploying agents not only as autonomous coding assistants, but as proactive collaborators capable of detecting and resolving external uncertainty arising from real-world underspecification.

## Limitations

**User Simulator** Our evaluation relies on an LLM-based user simulator to provide withheld information. While we implemented strict guardrails to prevent unintended leakage and generally observed reasonable simulator responses, recent studies highlight that LLM-simulated users can be un-

reliable proxies for human behavior, often being unnaturally cooperative and failing to reflect the nuance and variance of real human users (Naous et al., 2026; Seshadri et al., 2026). Results may therefore vary if real human users interact with the agents that we presented in this work.

**Prompting and Training** While our most successful approach relies on a multi-agent scaffold with tailored prompts for each agent, it demonstrates that frontier models possess the latent capacity to monitor their own uncertainty and proactively seek clarification out of the box. Rather than relying on prompting specialized agents, future work could explore utilizing these successful interaction trajectories to train single models with standard finetuning or reinforcement learning (RL) techniques to natively exhibit this calibrated, information-seeking behavior (e.g., Andukuri et al., 2024; Bhargava et al., 2024; Sun et al., 2026).

**Model Generalization and Cost** Our experiments were conducted using both a proprietary model (Claude Sonnet 4.5) and an open-weight model (Kimi K2.6), demonstrating that our multi-agent scaffold generalizes across model families. By relying on frontier models, our evaluation incurred a non-trivial financial cost (detailed in Appendix E). Furthermore, previous studies indicate that smaller open-weight models severely lack the calibration required for interactive underspecification (Vijayvargiya et al., 2026b), even with explicit prompting. The current findings may therefore be limited to frontier models and may not directly translate to smaller open-source models.

## Ethical Considerations

Our work investigated the ability of LLM agents to detect external uncertainty and seek clarification on underspecified software engineering tasks. While our results demonstrate that uncertainty-aware scaffolds can effectively resolve underspecified GitHub issues, SWE-bench Verified represents only a subset of software engineering tasks. As such, our empirical findings should not be extrapolated to suggest that agents will reliably detect missing information in other, often high-stakes environments, such as security-critical applications without further experimentation.

We also acknowledge the environmental and financial costs associated with the deployment of multi-agent systems. As detailed in our cost break-

down (Appendix E), the increased inference overhead in multi-agent systems is a trade-off that must be weighed against the gains in developer productivity.

**AI Use.** We used an AI assistant to assist with experimental code generation, and help improve the clarity and flow of the writing while revising the paper. However, all original writing, research conceptualization, methods, experiments, and analyses were performed by the authors, and any AI outputs were carefully verified.

## Acknowledgments

This work was supported by funding from the Vienna Science and Technology Fund (WWTF) through the project “Understanding Language in Context” (WWTF Vienna Research Group VRG23-007).

## References

- Mohammad Aliannejadi, Hamed Zamani, Fabio Crestani, and W. Bruce Croft. 2019. [Asking clarifying questions in open-domain information-seeking conversations](#). In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 475–484.
- Chinmaya Andukuri, Jan-Philipp Fränken, Tobias Gerstenberg, and Noah Goodman. 2024. [STaR-GATE: Teaching language models to ask clarifying questions](#). In *First Conference on Language Modeling (COLM)*.
- Anthropic. 2025. [Introducing Claude Sonnet 4.5](#). Accessed: 2026-03-15.
- Aman Bhargava, Cameron Witkowski, Alexander Dektov, and Matt Thomson. 2024. [Prompt baking](#). *arXiv preprint arXiv:2409.13697*.
- Maximillian Chen, Ruoxi Sun, Tomas Pfister, and Sercan O Arik. 2025. [Learning to clarify: Multi-turn conversations with action-based contrastive self-training](#). In *The Thirteenth International Conference on Learning Representations*.
- Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljube, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. 2024. [Introducing SWE-bench Verified](#). Accessed: 2026-03-15.
- Herbert H. Clark. 1996. *Using Language*. Cambridge University Press.
- Herbert H Clark and Deanna Wilkes-Gibbs. 1986. [Referring as a collaborative process](#). *Cognition*, 22(1):1–39.

- CMU LTI. 2024. Interactive SWE-bench: A dataset for software engineering issue resolution. <https://huggingface.co/datasets/cmu-lti/interactive-swe>.
- Harsh Darji and Thibaud Lutellier. 2025. Curiosity by design: An LLM-based coding assistant asking clarification questions. *arXiv preprint arXiv:2507.21285*.
- Robert X.D. Hawkins, Andreas Stuhlmüller, Judith Degen, and Noah D. Goodman. 2015. Why do you ask? Good questions provoke informative answers. In *Proceedings of the 37th Annual Meeting of the Cognitive Science Society*.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*.
- Bairu Hou, Yujian Liu, Kaizhi Qian, Jacob Andreas, Shiyu Chang, and Yang Zhang. 2024. Decomposing uncertainty for large language models through input clarification ensembling. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. GPT-4o system card. *arXiv preprint arXiv:2410.21276*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.
- Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, and 1 others. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.
- Sanyam Kapoor, Nate Gruver, Manley Roberts, Katie Collins, Arka Pal, Umang Bhatt, Adrian Weller, Samuel Dooley, Micah Goldblum, and Andrew G. Wilson. 2024. Large language models must be taught to know what they don't know. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Hyuhng Joon Kim, Youna Kim, Cheonbok Park, Junyeob Kim, Choonghyun Park, Kang Min Yoo, Sang-goo Lee, and Taeuk Kim. 2024. Aligning language models to explicitly handle ambiguity. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1989–2007, Miami, Florida, USA.
- Belinda Z. Li, Alex Tamkin, Noah Goodman, and Jacob Andreas. 2025. Eliciting human preferences with language models. In *The Thirteenth International Conference on Learning Representations*.
- Haau-Sing (Xiaocheng) Li, Mohsen Mesgar, André Martins, and Iryna Gurevych. 2023. Python code generation by asking clarification questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14287–14306, Toronto, Canada. Association for Computational Linguistics.
- Jialin Li, Yuan Wu, and Yi Chang. 2026a. ClarEval: A benchmark for evaluating clarification skills of code agents under ambiguous instructions. *arXiv preprint arXiv:2603.00187*.
- Xiaomin Li, Zhou Yu, Ziji Zhang, Yingying Zhuang, Swair Shah, Narayanan Sadagopan, and Anurag Benival. 2026b. Semantic volume: Quantifying and detecting both external and internal uncertainty in LLMs. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(37):31751–31759.
- Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Teaching models to express their uncertainty in words. *Transactions on Machine Learning Research*.
- METR. 2025. Research update: Algorithmic vs. holistic evaluation. Accessed: 2026-03-15.
- Moonshot AI. 2026. Kimi K2.6: Advancing open-source coding. Accessed: 2026-05-20.
- Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binqian Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. 2024. ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering*, 1(FSE):2332–2354.
- Tarek Naous, Philippe Laban, Wei Xu, and Jennifer Neville. 2026. Flipping the dialogue: Training and evaluating user language models. In *The Fourteenth International Conference on Learning Representations*.
- OpenAI. 2025. GPT-5.1: A smarter, more conversational ChatGPT. Accessed: 2026-03-15.
- Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*, pages 1–22.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186, Bangkok, Thailand. Association for Computational Linguistics.

- Sudha Rao and Hal Daumé III. 2018. [Learning to ask good questions: Ranking clarification questions using neural expected value of perfect information](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2737–2746, Melbourne, Australia. Association for Computational Linguistics.
- Preethi Seshadri, Samuel Cahyawijaya, Ayomide Odumakinde, Sameer Singh, and Seraphina Goldfarb-Tarrant. 2026. [Lost in simulation: LLM-simulated users are unreliable proxies for human users in agentic evaluations](#). In *Algorithmic Fairness Across Alignment Procedures and Agentic Systems*.
- Shannon Zejiang Shen, Valerie Chen, Ken Gu, Alexis Ross, Zixian Ma, Jillian Ross, Alex Gu, Chenglei Si, Wayne Chi, Andi Peng, Jocelyn J Shen, Ameet Talwalkar, Tongshuang Wu, and David Sontag. 2026. [Scaling collaborative effort with agents](#). In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 12254–12271, San Diego, California, United States. Association for Computational Linguistics.
- Weiwei Sun, Xuhui Zhou, Weihua Du, Xingyao Wang, Sean Welleck, Graham Neubig, Maarten Sap, and Yiming Yang. 2026. [Training proactive and personalized LLM agents](#). In *Third Conference on Language Modeling*.
- Alberto Testoni and Raquel Fernández. 2024. [Asking the right question at the right time: Human and model uncertainty guidance to ask clarification questions](#). In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 258–275, St. Julian’s, Malta.
- Sanidhya Vijayvargiya, Vijay Viswanathan, and Graham Neubig. 2026a. [Asking what matters: Reward-driven clarification for software engineering tasks](#). In *Forty-third International Conference on Machine Learning*.
- Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. 2026b. [Ambig-SWE: Interactive agents to overcome underspecificity in software engineering](#). In *The Fourteenth International Conference on Learning Representations*.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, and 1 others. 2025. [OpenHands: An open platform for AI software developers as generalist agents](#). In *The Thirteenth International Conference on Learning Representations*.
- Zora Zhiruo Wang, John Yang, Kilian Lieret, Alexa Tartaglioni, Valerie Chen, Yuxiang Wei, Zijian Wang Lingming Zhang, Karthik Narasimhan, Ludwig Schmidt, Graham Neubig, and 1 others. 2026. [Position: Humans are missing from AI coding agent research](#). Accessed: 2026-03-15.
- Jie JW Wu and Fatemeh H. Fard. 2025. [HumanEval-Comm: Benchmarking the communication competence of code generation for LLMs and LLM agents](#). *ACM Transactions on Software Engineering and Methodology*, 34(7):1–42.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R. Narasimhan. 2025.  [\$\tau\$ -bench: A benchmark for tool-agent-user interaction in real-world domains](#). In *The Thirteenth International Conference on Learning Representations*.
- Michael JQ Zhang and Eunsol Choi. 2025. [Clarify when necessary: Resolving ambiguity through interaction with LMs](#). In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 5541–5558, Albuquerque, New Mexico. Association for Computational Linguistics.
- Michael JQ Zhang, W. Bradley Knox, and Eunsol Choi. 2025. [Modeling future conversation turns to teach LLMs to ask clarifying questions](#). In *The Thirteenth International Conference on Learning Representations*.
- Ruiqi Zhong, Peter Zhang, Steve Li, Jinwoo Ahn, Dan Klein, and Jacob Steinhardt. 2023. [Goal driven discovery of distributional differences via language descriptions](#). In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Kaitlyn Zhou, Jena D. Hwang, Xiang Ren, and Maarten Sap. 2024. [Relying on the unreliable: The impact of language models’ reluctance to express uncertainty](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3623–3643, Bangkok, Thailand.
- Yifei Zhou, Song Jiang, Yuandong Tian, Jason Weston, Sergey Levine, Sainbayar Sukhbaatar, and Xian Li. 2025. [SWEET-RL: Training multi-turn LLM agents on collaborative reasoning tasks](#). *arXiv preprint arXiv:2503.15478*.

## A Dataset Construction and Validation

As mentioned in Section 3.1, we conduct our evaluation using an underspecified variant of SWE-bench Verified, directly adopting the dataset introduced by Vijayvargiya et al. (2026b). The authors automatically constructed this dataset through prompting GPT-4o to summarize the original GitHub issue. The prompt instructed the model to retain repository-specific terminology while removing important technical details.

To validate the synthetically generated dataset, Vijayvargiya et al. (2026b) applied distributional difference analysis (Zhong et al., 2023), an LLM-driven approach for discovering and describing differences between corpora, to compare the generated issues against naturally occurring underspecified issues in the original SWE-bench dataset. Their analysis found that natural issues typically retain more concrete technical details, such as code snippets, specific error messages, and file/line references, compared to the generated issues. Beyond this difference, however, the synthetic and natural issues were not found to be otherwise distinguishable. Consequently, the synthetic generation process produces instances that are often strictly *more* underspecified than natural occurrences. For our evaluation, this characteristic of the synthetic dataset is advantageous, making the task more challenging and heavily penalizing blind execution while encouraging proactive interaction.

**Empirical Spot-Check Analysis** To further verify the dataset’s validity for our evaluation, we conducted a manual qualitative analysis on a random sample of 10 task instances. For each instance, we compared the original, fully specified issue description with the synthetically generated underspecified issue description to determine whether the removed information was necessary for solving the task.

In 5 instances, the removed information was essential for solving the task, such as specific OS configurations or user-provided value constraints. In the remaining 5 instances, the summarized description retained sufficient technical details (e.g., specific class or method names) to likely allow an agent to successfully localize the relevant files in the repository requiring edits to correctly resolve the issue. Figure 7 illustrates one example of each case.

Notably, this variance closely mirrored actual agent performance. For instance, the HIDDEN baseline using Claude Sonnet 4.5 failed to solve all 5

instances where our manual analysis revealed that essential information was removed, but successfully solved the 5 instances where sufficient technical details were retained. This pattern is consistent with the significant performance gap between the HIDDEN and FULL baselines (54.80% vs. 70.80% for the Claude Sonnet 4.5 agent), highlighting that the dataset provides meaningful signal for when interaction is required to resolve underspecification.

## B Task and Scaffold Prompts

### B.1 Task Prompts

Figure 3 presents the SWE-bench task prompt template. The FULL baseline is provided with this prompt including the original, fully specified issue. The HIDDEN baseline and both UNCERTAINTY-AWARE scaffolds receive it with the underspecified issue. Conversely, only the INTERACTIVE BASELINE receives the augmented variant (highlighted in bold in Figure 3), which explicitly mentions that there is missing information and instructs the agent to first ask questions before proceeding.

### B.2 Agent Scaffold Prompts

Figure 4 shows specific prompts required for the uncertainty-aware agents.

For UNCERTAINTY-AWARE (SINGLE), we provide a recurring reminder prompt to the agent at each turn (Figure 4A) to assess for underspecification. If the agent detects underspecification, it is encouraged to use the `clarify` tool to query the user.

For UNCERTAINTY-AWARE (MULTI), we employ a customized system prompt for the Intent Agent (Figure 4B). This prompt instructs the agent to analyze the state history at each turn to detect underspecification. Rather than performing code edits, the Intent Agent is constrained to identifying missing information and determining when the Main Agent must pause execution to seek clarification from the simulated user.

To reduce the over-querying behavior observed with Kimi K2.6, we additionally experimented with an augmented Intent Agent prompt that instructs the agent not to flag underspecification when the user has already indicated they cannot provide further details. Specifically, we appended the following condition to the Intent Agent prompt “... or when the user cannot provide any further details or explicitly asks you to keep working on the task”. While this modification reduced the num-

### SWE-bench Task Prompt

```
<uploaded_files>
/workspace/{{ workspace_dir_name }}
</uploaded_files>
```

I've uploaded a python code repository in the directory {{ workspace\_dir\_name }}. Consider the following issue description:

```
<issue_description>
{{ instance.problem_statement }}
</issue_description>
```

Can you help me implement the necessary changes to the repository so that the requirements specified in the <issue\_description> are met?

I've already taken care of all changes to any of the test files described in the <issue\_description>. This means you DON'T have to modify the testing logic or any of the tests in any way!

Also the development Python environment is already set up for you (i.e., all dependencies already installed), so you don't need to install other packages. Your task is to make the minimal changes to non-test files in the /workspace/{{ workspace\_dir\_name }} directory to ensure the <issue\_description> is satisfied.

**I have not provided all the necessary details about the issue and I have some hidden details that are helpful. Please ask me specific questions using non-code commands to gather the relevant information that I have to help you solve the issue. Ensure you have all the details you require to solve the issue.**

**You have a limited number of turns. Do NOT interact with me more than three times so as to maximize the number of turns you have to work on the solution.**

Then, follow these phases to resolve the issue:

[...]

Figure 3: The base SWE-bench task prompt provided to the agents. The highlighted block in bold indicates the explicit clarification instructions added in the interactive variant, provided *only* to the INTERACTIVE BASELINE.

ber of times the agent asked a user while solving individual tasks, the agent continued to flag the vast majority of tasks as underspecified. This over-querying behavior, i.e., the tendency to ask questions even when it would be possible to solve the task without further clarification, suggests that the underlying Kimi K2.6 model has a limited ability to follow instructions and determine when tasks are underspecified.

### B.3 User Simulator Prompt

Figure 5 shows the prompt provided to the user simulator, adapted from the prompt in Vijayvargiya et al. (2026b). Initial experiments revealed that the simulated user occasionally provided misaligned guidance due to a lack of awareness regarding the specific constraints of our evaluation environment (i.e., OpenHands agent in SWE-bench). To prevent the simulator from misleading the agent or leaking unintended information, we augmented the original prompt with the following environment-specific guardrails:

- **Preventing test modifications (Rule 5):** In SWE-bench, test files should not be modified by the agent. Because the original simulated user lacked this context, it failed to correct the agent when it attempted to edit files inside

the /testbed directory. Rule 5 was added to instruct the simulated user to explicitly remind the agent of this if it asked about changing test files.

- **Resolving directory mismatches (Rule 6):** Agents in OpenHands operate within a designated /workspace directory, whereas SWE-bench imports and runs tests in a separate /testbed directory. Agents often became confused when changes in /workspace were not reflected in the tests run in /testbed. Rule 6 was added to instruct the simulated user to explicitly remind the agent of this if it asked why its edits weren't being reflected.

Additionally, to prevent unbounded interactions and encourage efficient information-seeking, the simulated user is restricted to three interaction turns per trajectory. After this limit is reached, it issues a canned response instructing the agent to proceed with the available information. Note that the INTERACTIVE BASELINE configuration is explicitly warned of this limit upfront, while our uncertainty-aware agents are not, testing their ability to independently seek information in an efficient manner.

## C Question Analyses

### C.1 Question Statistics

Table 2 provides a detailed breakdown of clarification-seeking behavior across the three interactive agent configurations using both LLM backbones.

Regarding the amount of interaction, the INTERACTIVE BASELINE initiates queries in nearly all tasks for Claude Sonnet 4.5 ( $N_{\text{ask}}=496$ , 1.02 Q/Task), but far fewer for Kimi K2.6 ( $N_{\text{ask}}=194$ , 1.00 Q/Task). This reduction is largely driven by instruction-following failures, where the model erroneously used the task completion tool to ask questions rather than initiating a proper query turn, resulting in many tasks where interaction was attempted but not successfully executed.

In general, the uncertainty-aware agents demonstrate more conservative query initiation but greater query volume. UA-MULTI initiated queries in 344 tasks for Claude Sonnet 4.5 and 435 tasks for Kimi K2.6, engaging in significantly more iterative dialogue (Claude: 3.06 Q/Task; Kimi: 8.71 Q/Task). The much higher query rate for Kimi K2.6 also reflects the model’s tendency to over-flag underspecification. UA-SINGLE initiated queries in more tasks with Claude Sonnet 4.5 ( $N_{\text{ask}}=369$ ) but in fewer tasks with Kimi K2.6 ( $N_{\text{ask}}=167$ ). For both models, the query rate was substantially lower than for the multi-agent scaffold (Claude: 1.84 Q/Task; Kimi: 1.23 Q/Task).

The length of these interactions also varies. For Claude Sonnet 4.5, the INTERACTIVE BASELINE generates highly verbose queries (251.33 tokens) and receives correspondingly long answers (415.53 tokens), reflecting a strategy to ask for all potentially missing information upfront. In contrast, UA-MULTI uses the most concise queries (171.57 tokens) and receives the shortest answers (173.65 tokens), supporting the observation that the multi-agent scaffold asks highly targeted, context-specific questions based on intermediate tool observations (e.g., see Figure 6). UA-SINGLE falls between the two (181.35/229.76 tokens). For Kimi K2.6, the pattern differs: UA-MULTI produces shorter queries (96.73 tokens) but receives markedly shorter answers (57.06 tokens), suggesting lower-quality clarification requests that elicit less informative responses. Notably, UA-SINGLE with Kimi K2.6 asks fewer but more substantive questions (143.18 tokens) and receives considerably longer answers (247.97 tokens).

Finally, we analyze the temporal distribution of these queries. We categorize the timing of each question based on its event ID position within the agent trajectory, divided into deciles: “Early” (1st–3rd decile), “Mid” (4th–7th decile), and “Late” (8th–10th decile). For Claude Sonnet 4.5, the INTERACTIVE BASELINE concentrates almost all questions in the Early stage (97.6%), while UA-SINGLE skews heavily Late (43.9%), suggesting it often attempts code modifications before recognizing an information gap. In contrast, UA-MULTI distributes queries across Early (41.8%) and Mid (43.4%) stages, reflecting continuous monitoring throughout the trajectory. For Kimi K2.6, the temporal patterns differ: UA-MULTI concentrates queries in the Mid stage (46.5%), while UA-SINGLE skews Early (51.0%). Notably, the INTERACTIVE BASELINE for Kimi K2.6 is heavily concentrated in the middle of the trajectory (66.5%), in contrast to the Early concentration seen with Claude Sonnet 4.5, likely reflecting the instruction-following failures that disrupted early-stage querying.

### C.2 Query Frequency by Task Difficulty

To evaluate whether agents appropriately calibrate their uncertainty to the complexity of the task, we mapped their querying behavior to the human-annotated difficulty levels provided by SWE-bench Verified (Chowdhury et al., 2024). Difficulty is categorized by the estimated time required for a human developer to fix the issue.

Table 3 presents the ask rates for both UA-SINGLE and UA-MULTI. For Claude Sonnet 4.5, UA-MULTI exhibits a greater difference in ask rates than UA-SINGLE, particularly between the “<15 min fix” and “15 min – 1 hour” levels (9.28% vs. 2.43% increase). Notably, both scaffolds exhibit an increased ask rate for the harder tasks with both agents reaching a 100% interaction rate on the most difficult “>4 hours” tasks. For Kimi K2.6, UA-MULTI queries the vast majority of tasks at every difficulty level (82.99%–100%) with a modest monotonic increase, although the consistently high query rate suggests reduced sensitivity to task complexity. UA-SINGLE displays a non-monotonic pattern, with ask rates declining from medium (36.02%) to harder tasks (30.95%), suggesting its query decisions do not reliably track task difficulty.

| Agent                    | Interaction Volume |               |              | Token Length |        | Timing Distribution (%) |      |      |
|--------------------------|--------------------|---------------|--------------|--------------|--------|-------------------------|------|------|
|                          | # Queried Tasks    | Total Queries | Avg # Q/Task | Avg Q        | Avg A  | Early                   | Mid  | Late |
| <i>Claude Sonnet 4.5</i> |                    |               |              |              |        |                         |      |      |
| UA-Multi                 | 344                | 1053          | 3.06         | 171.57       | 173.65 | 41.8                    | 43.4 | 14.8 |
| UA-Single                | 369                | 679           | 1.84         | 181.35       | 229.76 | 25.0                    | 31.1 | 43.9 |
| Interactive Baseline     | 496                | 508           | 1.02         | 251.33       | 415.53 | 97.6                    | 1.4  | 1.0  |
| <i>Kimi K2.6</i>         |                    |               |              |              |        |                         |      |      |
| UA-Multi                 | 435                | 3791          | 8.71         | 96.73        | 57.06  | 20.7                    | 46.5 | 32.8 |
| UA-Single                | 167                | 206           | 1.23         | 143.18       | 247.97 | 51.0                    | 32.5 | 16.5 |
| Interactive Baseline     | 194                | 194           | 1.00         | 176.63       | 244.07 | 28.4                    | 66.5 | 5.2  |

Table 2: Question statistics across evaluation settings. # *Queried Tasks* denotes the number of tasks where the agent initiated at least one query. Timing distributions are categorized into Early (1st–3rd decile), Mid (4th–7th decile), and Late (8th–10th decile) based on the question’s event ID position within the trajectory.

| Difficulty Level         | # Tasks | Ask Rate (%) |          |
|--------------------------|---------|--------------|----------|
|                          |         | UA-SINGLE    | UA-MULTI |
| <i>Claude Sonnet 4.5</i> |         |              |          |
| <15 min fix              | 194     | 71.13        | 62.37    |
| 15 min–1 hour            | 261     | 73.56        | 71.65    |
| 1–4 hours                | 42      | 85.71        | 78.57    |
| >4 hours                 | 3       | 100.00       | 100.00   |
| <i>Kimi K2.6</i>         |         |              |          |
| <15 min fix              | 194     | 29.90        | 82.99    |
| 15 min–1 hour            | 261     | 36.02        | 88.89    |
| 1–4 hours                | 42      | 30.95        | 92.86    |
| >4 hours                 | 3       | 66.67        | 100.00   |

| Evaluation Setting                                                        | Resolve Rate (%) |              |
|---------------------------------------------------------------------------|------------------|--------------|
|                                                                           | Asked            | Did Not Ask  |
| <i>Claude Sonnet 4.5</i> ( $N_{\text{ask}} = 369, N_{\text{not}} = 131$ ) |                  |              |
| UA-Single                                                                 | 58.54            | 68.70        |
| Full                                                                      | 67.48            | <b>80.15</b> |
| Hidden                                                                    | 47.43            | 75.57        |
| UA-Multi                                                                  | 66.40            | 77.86        |
| Interactive Baseline                                                      | <b>68.29</b>     | 76.34        |
| <i>Kimi K2.6</i> ( $N_{\text{ask}} = 167, N_{\text{not}} = 333$ )         |                  |              |
| UA-Single                                                                 | 60.48            | 62.16        |
| Full                                                                      | <b>69.46</b>     | <b>74.47</b> |
| Hidden                                                                    | 35.33            | 65.47        |
| UA-Multi                                                                  | 65.87            | 71.17        |
| Interactive Baseline                                                      | 37.72            | 51.95        |

Table 3: Ask rates by SWE-bench Verified difficulty (es-

Table 3: Ask rates by SWE-bench Verified difficulty (estimated time-to-fix) for uncertainty-aware (UA) agents.

### C.3 Conditional Resolve Rates for UNCERTAINTY-AWARE (SINGLE)

Table 4 details the task resolve rates conditioned on whether the UA-SINGLE agent chose to interact with the user. For Claude Sonnet 4.5, when UA-SINGLE chose to ask questions (369 of 500 tasks), it successfully elicited missing information, resolving 216 tasks compared to only 175 for the HIDDEN baseline on the same subset. Conversely, on the 131 tasks where the agent refrained from interaction, its performance closely mirrored the HIDDEN baseline (90 vs. 99 resolved tasks). These results indicate that UA-SINGLE with Claude Sonnet 4.5 can effectively detect when missing information is critical for task resolution, although the performance gap with other configurations suggests that its clarification-seeking behavior remains less calibrated compared to UA-MULTI. For Kimi K2.6, the pattern is weaker: resolve rates are nearly identical regardless of whether the agent asked (60.48%) or did not (62.16%), suggesting more weakly calibrated clarification-seeking behavior.

Table 4: Resolve rates (%) conditioned on whether the UA-SINGLE agent queried the user at least once.  $N_{\text{ask}}$  and  $N_{\text{not}}$  denote subset sizes. Best per-subset results in **bold**.

### C.4 Qualitative Example of Agent Interaction

The transcripts in Figure 6 illustrate the distinct information-seeking strategies employed by the agents using Claude Sonnet 4.5 for the `pytest-dev__pytest-7324` task, which involves resolving a pytest-related Python interpreter crash. As highlighted in bold, while the baseline relies on more generic questions, both uncertainty-aware agents ask more specific, technical questions. In particular, note how the multi-agent scaffold uniquely engages in a mid-trajectory follow-up query after observing a test failure.

## D Underspecified Issue Examples

Figure 7 provides a comparison of the original and underspecified issue descriptions for two tasks. In creating the underspecified examples, the summarization process generally targets the removal of specific details such as code snippets, file paths/-

line references and stack traces, while preserving relevant terminology to describe the issue at a high level. However, as noted in our spot-check analysis (Appendix A), the extent of this reduction varies across instances. In some cases, the retained high-level description still provides sufficient signal for an agent to localize the relevant code, while in others the removed details are essential for correct resolution, providing meaningful signal for when clarification-seeking behavior is required.

## E Computing Costs

Table 5 details the total inference costs for each evaluation setting across both LLM backbones. For Claude Sonnet 4.5, all agent inference calls were executed via the Anthropic API. While the multi-agent scaffold more than doubles the total inference cost compared to the single-agent baselines, the absolute financial cost per task remains negligible. We argue that this increase in compute expenditure is a highly favorable trade-off; by proactively resolving underspecification and significantly increasing the overall task resolve rate, the multi-agent setup ultimately saves substantial human developer time and effort that would otherwise be spent on debugging solutions that are misaligned with the original intent. For Kimi K2.6, all agent inference calls were executed via the Moonshot API. Kimi K2.6 runs at roughly 30–50% of the cost of Claude Sonnet 4.5, offering substantial cost efficiency gains while maintaining competitive performance. Additionally, the multi-agent scaffold has a total inference cost less than double that of the single-agent baselines, making Kimi K2.6 a cost-efficient alternative for deploying the framework.

| Setting              | Claude Sonnet 4.5 |                      | Kimi K2.6       |                      |
|----------------------|-------------------|----------------------|-----------------|----------------------|
|                      | Total Cost (\$)   | Avg Cost / Task (\$) | Total Cost (\$) | Avg Cost / Task (\$) |
| Full                 | 817.02            | 1.63                 | 316.18          | 0.63                 |
| Hidden               | 899.43            | 1.80                 | 418.26          | 0.84                 |
| Interactive Baseline | 697.88            | 1.40                 | 270.42          | 0.54                 |
| UA-Single            | 1017.34           | 2.03                 | 323.27          | 0.65                 |
| UA-Multi             | 1748.08           | 3.50                 | 498.36          | 1.00                 |

Table 5: API costs (in USD) across evaluation settings on SWE-bench Verified.

## Uncertainty-Aware Agent Scaffold Prompts

### A. UNCERTAINTY-AWARE (SINGLE) Reminder Prompt

Before selecting your next tool or drafting a reply, first reason explicitly about whether you have enough information. After stating that reasoning, write a line formatted as Ambiguity assessment: <clear|ambiguous>. If the assessment is ambiguous, immediately call the clarify tool with questions for the user. If it is clear, proceed normally.

### B. UNCERTAINTY-AWARE (MULTI) Intent Agent Prompt

You are Intent Agent, a specialized OpenHands agent. Your sole purpose is to assess at a given turn whether there exists ambiguity or key missing information which may impact the work of the main agent trying to solve a user's task.

#### <ROLE>

- Your primary role is to interpret, at any given point, the conversation history so far. This includes user messages, as well as main agent actions and observations.
- Keep in mind these questions when you are performing the analysis:
  1. What is the intended goal?
  2. What is missing, ambiguous, or assumption-heavy?
  3. What must be clarified before any further code or tool executions can happen?

#### </ROLE>

#### <WORKFLOW>

1. Carefully read the provided conversation history, including user messages, main agent actions, and observations.
2. Enumerate any ambiguities, missing information, or assumptions that could impact the main agent's ability to successfully complete the user's task.
3. Finally, call the clarify\_decision tool, producing a structured JSON output only indicating with a boolean whether or not clarification is required ('needs\_clarification'), and providing the reasons if necessary. if the main agent attempts to read a file that does not exist, you would output something like:

```
{
  'needs_clarification': true,
  'reasons': 'The main agent attempted to read a file that does not exist,
             indicating possible misunderstanding of the file structure.'
}
```

#### </WORKFLOW>

#### <EXTRA\_GUIDELINES>

- Be clear and concise in your reasoning, avoiding unnecessary complexity. Keep your reasoning short, only 30-50 words.
- You will be provided with a summary of the conversation history so far, including user messages, main agent actions, and observations. Some of these may be truncated with "...".
- If you ever see a user message of the form "1a, 2b, 3c", this refers to answers for the main agent's previous clarifying questions.

#### </EXTRA\_GUIDELINES>

Figure 4: Prompts required for the custom uncertainty-aware agent scaffolds. Part A shows the reminder prompt for the UNCERTAINTY-AWARE (SINGLE) agent at each turn. Part B shows the system prompt for the specialized Intent Agent in the UNCERTAINTY-AWARE (MULTI) agent scaffold.

### User Simulator Prompt

You are a GitHub user reporting an issue. Here are the details of your issue and environment:

Issue: {issue}

Files relative to your current directory: {files}

Your task is to respond to questions from a coder who is trying to solve your issue. The coder has a summarized version of the issue you have. Follow these rules:

1. If the coder asks a question that is directly related to the information in the issue you have, provide that information.
2. Always stay in character as a user reporting an issue, not as an AI assistant.
3. Keep your responses concise and to the point.
4. The coder has limited turns to solve the issue. Do not interact with the coder beyond 3 turns.
5. If the coder asks about modifying test files, remind them that all changes to any of the test files described in the issue are already handled and should not be further modified.
6. If the coder says that their fixes aren't reflected in tests or asks why the expected test behavior isn't changing, remind them that /testbed is where tests are imported and run, but they should make their final edits in /workspace.

Respond with "I don't have that information" if the question is unrelated or you're unsure.

Figure 5: The prompt provided to the user simulator, including specific guardrails to prevent unintended test modifications (rule 5) and resolve environment directory mismatches (rule 6).

### INTERACTIVE BASELINE

**Agent:** Based on the issue description mentioning “Python expression”, “assertion failure in the Python compiler”, and “constant names”, this is likely related to pytest’s assertion rewriting mechanism. Let me ask specific questions:

1. What is the **specific Python expression** that causes the crash?
2. What is the **exact error message** or assertion failure from the Python compiler? Is there a stack trace or specific assertion message that appears?
3. Is there a **specific Python bug report** or bpo issue number that this is related to? The description mentions “a known issue” - what is the reference for this?

.....  
**User:**

1. The minimal reproducer is this call into pytest’s mark expression compiler: `Expression.compile("False")`. Just compiling the string "False" (as a mark expression) is enough to trigger the crash on a debug build of Python 3.8+.
2. The crash is an assertion failure in the CPython compiler. On a debug build, it looks like:  

```
python: Python/compile.c:3559: compiler_nameop: Assertion
    !_PyUnicode_EqualToASCIIString(name, "None") &&
    !_PyUnicode_EqualToASCIIString(name, "True") &&
    !_PyUnicode_EqualToASCIIString(name, "False")' failed.
[1] 29440 abort (core dumped) python
```
3. The related CPython bug report is bpo-40870: <https://bugs.python.org/issue40870>. That’s the known issue aimed at improving this behavior I was referring to.

### UNCERTAINTY-AWARE (SINGLE)

**Agent:** I need some clarification before proceeding:

1. What is the specific Python expression that causes the interpreter to crash on Python 3.8+ debug builds? For example, is it **related to assertions, specific constant values, or particular syntax patterns**?
2. Which constant names are problematic and cause the assertion failure in the Python compiler? For example, are they names like, `'__debug__'`, `'True'`, `'False'`, `'None'`, or other special constants?
3. You mentioned you’ve already taken care of test file changes. Are these **test files already committed in the repository**, or should I look for them in a specific location? Can you point me to the test file that demonstrates this issue?
4. Is there a **specific GitHub issue number** or Python bug tracker reference (bpo number) for the ‘known issue’ you mentioned that is related to this problem?

.....  
**User:**

1. The minimal expression is just using `Expression.compile("False")` (the string "False" as the expression). That alone triggers the crash on a Python 3.8+ debug build.
2. The problematic constant names are None, True, and False. The assertion in `compiler_nameop` explicitly checks that the name is not any of those three.
3. The tests for this issue are already handled; you shouldn’t modify any test files further.
4. Yes, the related Python bug tracker reference is bpo-40870 (<https://bugs.python.org/issue40870>).

Figure 6: Comparison of user-agent interactions on the `pytest-dev__pytest-7324` task (Claude Sonnet 4.5). Both the INTERACTIVE BASELINE and the UNCERTAINTY-AWARE (SINGLE) agent successfully identify missing information but group all their queries into a single, upfront interaction turn. Bold text highlights INTERACTIVE BASELINE’s generic queries versus the more specific, technical queries of UNCERTAINTY-AWARE (SINGLE).

## UNCERTAINTY-AWARE (MULTI)

**Agent (Turn 1 - Early Stage):** I need more specific information to understand and fix the issue properly:

1. Can you provide the specific Python expression that causes the crash? For example, is it something like **assert \_\_debug\_\_** or a specific constant name?
2. Do you have the **actual error message** or assertion failure from Python's compiler/interpreter that appears on a debug build?
3. Is there a **specific test file or test case** that was added to reproduce this issue?
4. What is the **'known issue'** this is linked to? Is it a Python bug tracker issue or a GitHub issue?

---

**User:**

1. The specific expression is just `Expression.compile("False")`, i.e. the string "False" being compiled by `Expression.compile` in `src/_pytest/mark/expression.py`.
2. Yes, on a debug build it crashes with:  

```
python: Python/compile.c:3559: compiler_nameop: Assertion
`!_PyUnicode_EqualToASCIIString(name, "None") &&
 !_PyUnicode_EqualToASCIIString(name, "True") &&
 !_PyUnicode_EqualToASCIIString(name, "False")' failed.
[1] 29440 abort (core dumped) python
```
3. There is a test covering this scenario, but all changes to the test files are already handled and shouldn't be modified further for this task.
4. The linked known issue is a Python bug tracker issue: bpo-40870 (<https://bugs.python.org/issue40870>).

---

**Agent (Turn 2 - Mid Stage, Execution Phase):** The **test still fails after modifying the source code**. This suggests either a caching issue or the module hasn't been reloaded. Let me check if the changes were saved correctly and try reloading.

1. **Should I continue debugging** why the changes aren't taking effect?

---

**User:** No, you don't need to keep debugging that part. Remember that the tests are imported and run from `/testbed`, but your final code changes should be made in `/workspace`. If your fixes aren't reflected in the tests, it's likely due to that setup rather than a caching issue.

Figure 6: (continued). Unlike the single-agent setups, the multi-agent framework successfully engages in additional late-stage, iterative clarification following a test failure on the `pytest-dev__pytest-7324` task.

| pylint-dev__pylint-7080                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Original:</b></p> <p><b>Bug description:</b> When running recursively, it seems ignore-paths in my settings in pyproject.toml is completely ignored.</p> <p><b>Configuration:</b></p> <pre>[tool.pylint.MASTER] ignore-paths = [     # Auto generated     "^src/gen/.*\$", ]</pre> <p><b>Command used:</b></p> <pre>pylint --recursive=y src/</pre> <p><b>Pylint output:</b></p> <pre>***** Module region_selection src\gen\about.py:2:0: R2044: Line with empty comment (empty-comment)  [...]</pre> <p><b>Expected behavior:</b> src\gen\* should not be checked.</p> <p><b>OS / Environment:</b> Windows 10.0.19044</p> |
| <p><b>Underspecified:</b> There is an issue with a tool configuration where a recursive option is not respecting specified ignore paths, leading to unintended files being processed. This occurs despite the paths being correctly set in the configuration file. The problem is observed when running a specific command, and the expected behavior is for certain directories to be excluded from processing.</p>                                                                                                                                                                                                             |
| <p><b>Qualitative Analysis:</b> Omits the OS (Windows 10.0.19044) and Pylint output with backslash paths (src\gen\about.py), which contains the key information that the issue is occurring on a Windows filesystem.</p>                                                                                                                                                                                                                                                                                                                                                                                                         |

| scikit-learn__scikit-learn-26323                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Original:</b> When using set_output on a ColumnTransformer, it sets the output on its sub-transformers but ignores the transformer defined in remainder.</p> <p><b>Steps to reproduce:</b></p> <pre>import pandas as pd from sklearn.compose import make_column_selector, make_column_transformer from sklearn.feature_selection import VarianceThreshold  df = pd.DataFrame({"a": [True, False, True], "b": [1, 2, 3]}) out1 = make_column_transformer(     (VarianceThreshold(), make_column_selector(dtype_include=bool)),     remainder=VarianceThreshold(),     verbose_feature_names_out=False, ).set_output(transform="pandas").fit_transform(df) print(out1)</pre> <p><b>Expected/Actual results:</b> [...]</p> <p><b>Versions:</b> sklearn 1.2.2, pandas 2.0.1, Python 3.10.6, Linux-5.15.0-71-generic-x86_64</p> |
| <p><b>Underspecified:</b> There is an issue with ColumnTransformer in scikit-learn where the set_output method does not apply correctly to the transformer specified in the remainder parameter when it is an estimator. This results in incorrect output when the transformation is applied, as the remainder transformer is not included in the final result as expected.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <p><b>Qualitative Analysis:</b> Preserves key class (ColumnTransformer), method (set_output), and parameter (remainder) names, so an agent can locate the relevant code without clarification.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

Figure 7: Comparison of original (fully specified) and synthetically underspecified descriptions for two evaluation instances. In pylint-dev\_\_pylint-7080 (top), the missing OS information renders completion without clarification challenging. In scikit-learn\_\_scikit-learn-26323 (bottom), preserved class and method names are sufficient for an agent to localize and resolve the issue autonomously.